

# Un Standard

Luc Alexander Lüthi

January 30th, 2026

## Contents

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>Changes Since Experimentation Began</b> | <b>1</b> |
| <b>2</b> | <b>Current Documentation</b>               | <b>1</b> |
| 2.1      | Special Forms . . . . .                    | 2        |
| 2.2      | Base IR . . . . .                          | 3        |
| 2.3      | Examples . . . . .                         | 4        |
| <b>3</b> | <b>Standard Library</b>                    | <b>6</b> |
| <b>4</b> | <b>Standard Constraint System</b>          | <b>6</b> |

## 1 Changes Since Experimentation Began

Register coloring was taking place with a linear allocator, this was done to get the project out the door but it is commonly known that linear allocators are insufficient for any model of computation with arbitrary jumps between blocks. The linear allocator was replaced with a control flow graph based allocator which is not an optimal implementation but produces correct bytecode. Further, the (err "message") special form was taking place at compile time as it was processed in substrate time, so any time it was descended over it would trigger. It was moved to be an interrupt in the bytecode, allowing it to be runtime and compile time optional, and can now be invoked with a simple wrapper bind by the same name.

```
(bind err (message)
  ((uid (syscall x) (
    (reg x)
    (reg syscall)
    (reify x message)
    (mov syscall 1)
    (int syscall)
    (mov syscall 3)
    (int syscall))))))

(bind main ()
  ((err "this is the message to output on crash")))
```

## 2 Current Documentation

Un is an experimental meta compiler substrate research project. A few things to note:

The implementation is not optimal, and it does not produce optimal bytecode.

The implementation is research grade, not industry grade, it currently does not produce native binaries, although the resulting bytecode can be trivially translated to a target architecture of choice if you the user desires.

The language is not meant to be used as is, it's a backend + an experimental set of metacomputation structures for which the user is meant to implement a compiler frontend to utilize the experimental features.

The key feature being experimented with is language constraints as first class structures, everything else is secondary and is only implemented in support of or as a base for this feature. Do not expect the data structures or algorithms used in these cases to be flawless, they are meant to be minimal.

The expected use case for this tool is experimentation with novel language features, and it is built to promote fast iteration times. The process of writing constraints like type systems, borrow checkers, capability systems, proof engines, and effect systems is being optimized, but other areas of compiler research are possible. Register allocation and lowering is handled, and the target is a 64 bit bytecode VM meant to be as tiny as possible while faithfully emulating how modern CPU architectures typically work. This VM is missing some interrupts right now, but is fully capable of concurrent programming. Tokenization and parsing have been ignored, as they are trivial endeavors.

The basic premise is that the language provides an intermediate representation, and a series of simple meta primitives to compose the instructions in this representation to infinitely extend the surface area of the language. There is full reification and reflection between the high level s expression environment used for meta computation, and the runtime bytecode environment.

As you write a program in this language, you are acting as a purpose built compiler. This is compilation as computation. If you imagine a compiler as a set of all admissible programs, then the more constraint you add the smaller that set becomes, until theoretically you have a compiler which admits only the one correct program for the task at hand, a program which is built to describe not how a program performs its task, but the constraints which bind what the program must achieve. This can include but is not limited to: what task metric must be completed, how much memory the program uses, the type safety of the program, the borrow safety of the program, whether the program is representable by a finite state automaton, whether the program has certain provable invariants, whether the program is permission safe. This is logic programming, but constraint based.

## 2.1 Special Forms

`(bind name args body)`

binds a name to a compile time syntactic expansion.

`(comp vm_name (body))`

computes a body expression block at compile time in the provided virtual machine. The first 100 bytes of the virtual machine + the program section are ephemeral per invocation, but the rest of the memory space is entirely persistent between invocations. This enables compile time reasoning about the program on a global level. Any register variables found in the body will be reified into the ephemeral static section of the provided VM and will be replaced inline with the address into the machines memory space where that value is stored. Values are localized slices comprising of a length followed by a buffer of data. The final expression of a block will be reflected back to the s expression substrate in the form of an s expression,

converting a slice buffer into symbols, s expressions and values. These symbols and expressions can be stored in the VM and reflected back in a differing invocation.

`(uid list body)`

defines a list of unique identifiers which will be abstracted away to unique tokens before lowering to a normalized IR. These aliases only apply in the provided body.

`(flat list)`

flattens an s expression. `(f (flat (a b c))) -> (f a b c)`

`(use path)`

searches for the file in path and compiles the binds found in that file, replacing the invocation with the expression body of that module. This allows for imports as well as external compile time execution.

This is not a Lisp. Invoking a user defined bind will inline that procedure not call it like a function.

## 2.2 Base IR

```
(reg x)
(mov x (at x))
(mov (at x) x)
(mov x ffff)
(mov (at x) ffff)
```

```
(add x x 7)
(add x 1 4)
```

Binary ops: add sub mul div mod uadd usub umul udiv umod and or xor shl shr Unary ops: not com

```
(reg x)
(mov x 0)
(label loop)
  (cmp x 5)
  (jge exit)
  (add x x 1)
  (jmp label)
(label exit)
```

You can jump with jmp, jeq, jne, jlt, jgt, jle, jge Calls work with (call name) and do stack manipulation

```
(psh x)
(pop x)
```

```
(ret 7)
(ret x)
```

Reify higher level structures with reif:

```
(reif x (a b c))
(reif x "hello")
(reif x ffffffff)
\end{verbatim}
```

```

        create unique identifiers with (uid list block)

\begin{Verbatim}[tabsize=4]
(uid (loop i) (
  (reg i)
  (mov i 0)
  (label loop)
  ...
))

```

You can nest expressions as long as they have a terminal symbol

```

(mov x (
  (reg y)
  (mov y 7)
  (y)
))

```

## 2.3 Examples

examples/std.un provides a simple standard library with control flow, an arena allocator, a basic growable buffer, and a working constraint manager.

```

(use "std.un")

(constraint vacuous_false key val (
  (err "compile time error")
))

(constraint vacuous_true key val (
  (nop)
))

(bind main ()
  ((uid (i address x y pool subpool) (
    (reg pool)
    (reg subpool)
    (reg address)
    (reify address 00010000)
    (mov pool (arena address 4000))
    (mov subpool (subarena pool 1000))
    (reg x)
    (reg y)
    (if (subpool) (
      (mov x (alloc subpool 8))
      (if (x) (
        (print "alloc")
      ))(
        (print "fail")
      ))
    ))(
      (print "subfail")
    )
  ))

```

```

    ))
    (mov x (buffer subpool F0))
    (for i 0 10 (
        (mov y (append subpool x i))
        (print "appended")
    ))
    (constraint_setup)
    (constrain vacuous_true)
    (info "a" "b")
    (mov x 0)
    (int x))))))

(main)

```

examples/forth.un provides some basic primitives for a forthlike language + a bind for word definitions.

```

(use "forth.un")

```

```

(word double (
    (dup)
    (psh 2)
    (*))
)

```

```

(word sqr (
    (dup)
    (*))
)

```

```

(bind main () (
    (uid (x) (
        (reg x)
        (mov x 1)
        (psh x)
        (dup)
        (ovr)
        (+)
        (mov x 0)
        (int x)
    ))
))

```

```

(main)

```

### 3 Standard Library

The standard library for un is meant to be very simple and minimal. This is not an opinionated language ecosystem, and the API of the standard reflects that. Extensions to the standard can be made to accommodate any desired feature in the target frontend language as needed.

## 4 Standard Constraint System

The entire standard library is riddled with constraint information hooks, these hooks can be added to any portion of user code for custom constraints. A set of simple constraints is to be provided in the standard library.